Library WordExcel

The WordExcel class has been conceived as a library of functions (methods) that support the C# programmer and that would save a large number of programming lines. On the other hand, it greatly reduces the loss of time produced by the "how to do it" in programming time. This document is basically oriented to the neophyte C# programmer and for this reason we try to be as explicit as possible in the document.

This class is represented by the dynamic library WE.DLL attached to this project. In order to work with the methods of the WE class, the following steps must be followed:

Main and Mandatory Step:

To work with Word or Excel you must add the NuGet package ClosedXML (which includes the official Microsoft SDK DocumentFormat.OpenXml), as
described later at the beginning of the section in graphic form:

==>**METHODS FOR GENERATION OF WORD DOCUMENTS**

This is done in Visual Studio 2022 in:

Tools->Nuget Package Manager->Package Management Console

and when the PM> prompt appears you must type:

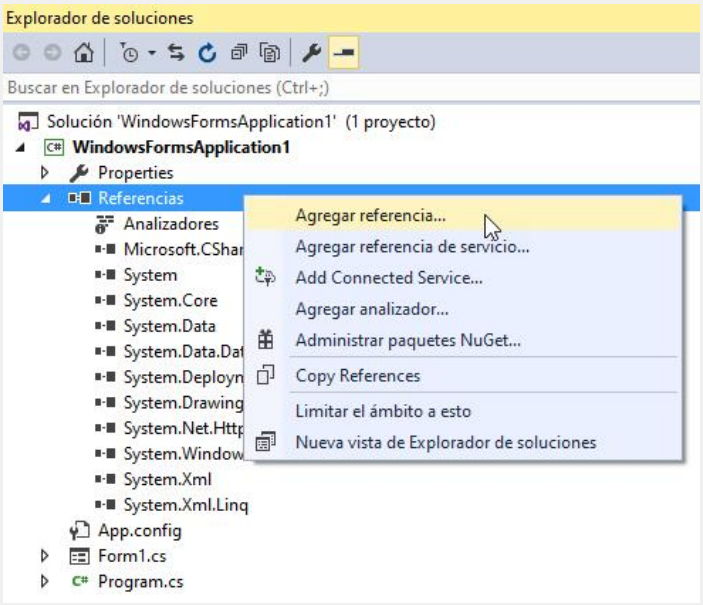
Install-Package ClosedXML -Version 0.102.2 (then wait for this process to finish)

This same package works for both Word and Excel, so there is nothing else to install.

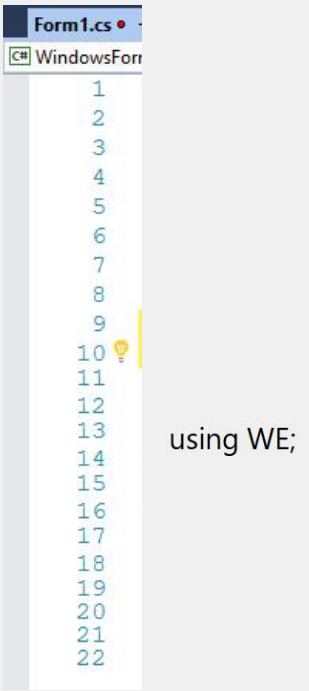
1) Then we start by adding a reference to said DLL (WE.DLL)
that remains after installation, on the desktop screen. Its convenient that
Immediately after installation, you copy said DLL to a personal folder in order to always add references from that custom place, since the desktop is
unsafe and at any time said DLL can be deleted by mistake and this
would imply the need to reinstall the application. We suggest the same with the
user manuals (In Spanish and English) and the files associated with the example program, which must be placed in the root of the D:\ drive, for it to run correctly.

Important note:

If you also want to add the multifunctional library LN.DLL, then it is not necessary
add no Package of type Nuget, just add it as a normal reference.
At the end of the installation of LN.DLL it is also on the desktop and for this reason we recommend the same as for WE.DLL, copy it to your personalized folder.

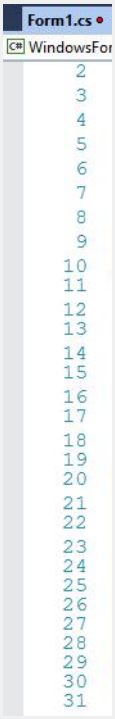


2) We then add the WE namespace using the corresponding using within the program code in the traditional way:



using WE;

3) From this moment you can instantiate the object with the desired name (in our case, called OBJ):



4) Now when you write OBJ followed by a period, the complete list of all the methods of said class will quickly appear, and all you have to do is select the desired method.

The WE class is fundamentally designed for Winforms type applications (.exe), but the document clearly specifies which methods can be used in WEB applications and which in Winforms applications.

WordExcel CLASS METHODS

METHODS FOR GENERATION OF WORD DOCUMENTS 5
Word methods 8
Method of Opening and Creating a Word Document 8
Method of Creating a Heading of type Title1, Title2, etc. 9
Word Document Closing Method 11
Method of Inserting an Image into the Word Document 11
Table Creation Method 12
Columnar Header Creation Method 13
Header Creation Method with Italic Letters 14
Cell Content Writing Method 14
Header and Footer Methods 15
Footer Creation Method 16
Method of Adding a Paragraph to the Document 17
Example Program for Creating a Word Document 18
Generated .docx document 21
Excel methods 22
ExcelCreateDocExcel() Method for Creating an EXCEL Document with its Initial Sheet 22
METHOD ExcelOpenFile(string PathFile) 23
(Opening an Excel Document) 23
ExcelSaveFile() METHOD TO SAVE THE EXCEL DOCUMENT 23
ExcelFormatTitles() METHOD OF WRITING A COLUMN TITLE (Row = 1, Column = n) 24
ExcelFormatCell() METHOD OF FORMATTING A CELL 25
ExcelWriteCellRC() METHOD OF WRITING A CELL IN AN EXCEL DOCUMENT 26
ExcelWriteCell() METHOD OF WRITING A CELL OF THE EXCEL DOCUMENT 26
ExcelGetCellRC() METHOD OF WRITING A CELL WITH A NAME IN string 27
""ExcelGetCell" METHOD FOR READING A CELL OF THE EXCEL DOCUMENT 27
ExcelGetCellDataTypeRC() METHOD OF READING THE DATA TYPE OF A CELL (r, c) 28
ExcelGetCellDataType() METHOD FOR READING THE DATA TYPE OF A CELL IN "string" FORMAT 28
ExcelFormula() METHOD TO DESCRIBE A FORMULA IN EXCEL 29
Example program for generating an EXCEL document 30
DataGridView METHODS 34
DGVColumnSumarize() method 34
DGVColumnSumarizeSelectedRows() method 34
DGVColumnAverage() method 35
DGVColumnAverageSelectedRows() method 35
DGVGetCell() method 36
DGVPutCell() method 36

METHODS FOR GENERATION OF WORD DOCUMENTS

The following explains in detail a set of methods developed to make it easier for the C# programmer to generate a Word document (.docx, .doc).

To carry them out we have chosen the official Microsoft SDK for Office documents, the NuGet package DocumentFormat.OpenXml (version 2.16.0), which is distributed under the MIT license and allows free use, including in commercial applications. Until October 2026 these methods used the Xceed Docx package, whose license only allows non-commercial use, and that is why it was replaced.

The Open XML SDK has been developed by Microsoft; its documentation and source code are at:

https://learn.microsoft.com/en-us/office/open-xml/open-xml-sdk (Documentation)
https://github.com/dotnet/Open-XML-SDK (Source code and MIT license)

The advantage that I have observed in it is that with this package you can do this type of work without having to have Microsoft Office installed on your PC, in my case personal where I use Ashampoo Write, which is an equivalent to Microsoft's Word, it worked perfectly and it is assumed that it will work in the same way in all those products that are used to create .docx documents compatible with MSWord.

If the user has Microsoft Office installed, then they can access Word as shown below:

```
//=========================================================
using System;using System.IO;
//Interop is not located if we do not have Microsoft Office installed
using Microsoft.Office.Interop.Word;
class Program{static void Main()
        {
        // Create an instance of the application WordApplication
        wordApp = new Application();
        // With the Application class the new app is instantiated
        // * * * Create a new program
//=========================================================
```

In my personal opinion, the Microsoft SDK is very complete but also very detailed: to write a simple formatted paragraph you have to build several document elements by hand, and at the beginning there is a bit of resistance to breaking the inertia and starting to program fluently and in a simple way.

Based on what I said above, I have developed a set of methods with clear and simple logic, which work perfectly and help save programming time.

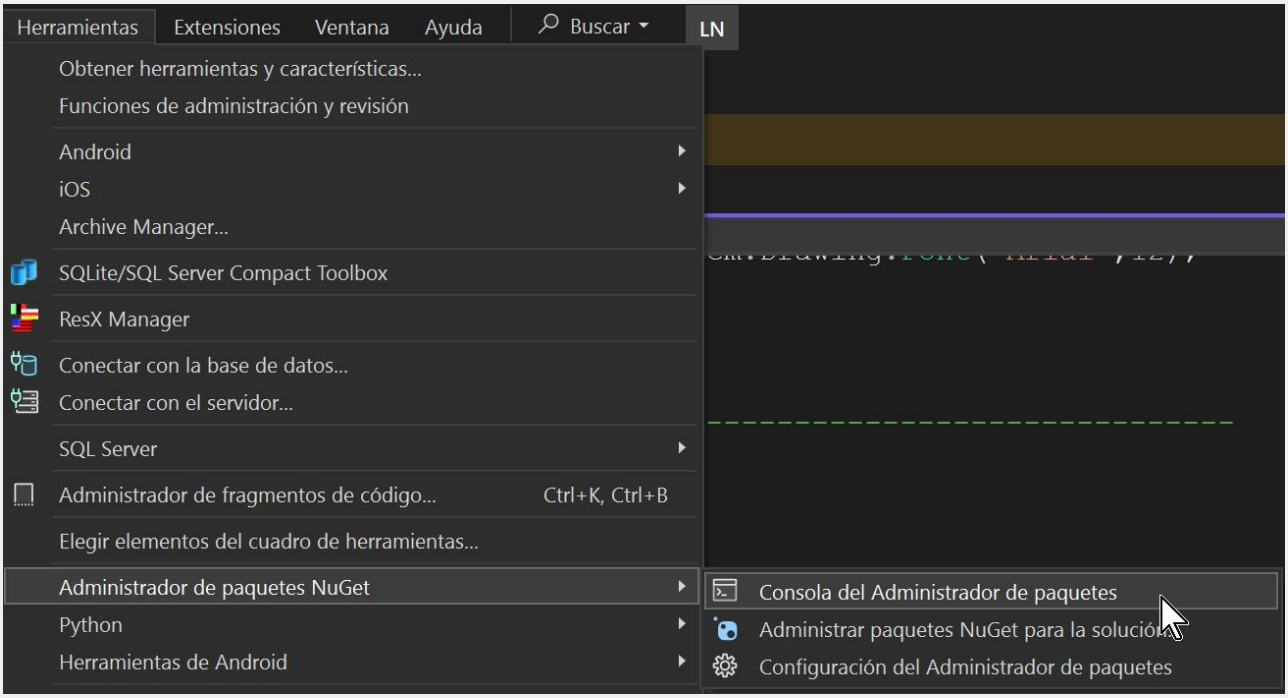
Together with this document I attach an example that uses all the methods developed.

It was programmed with:

Microsoft Visual Studio Community 2022 (64-bit)
Version 17.8.3

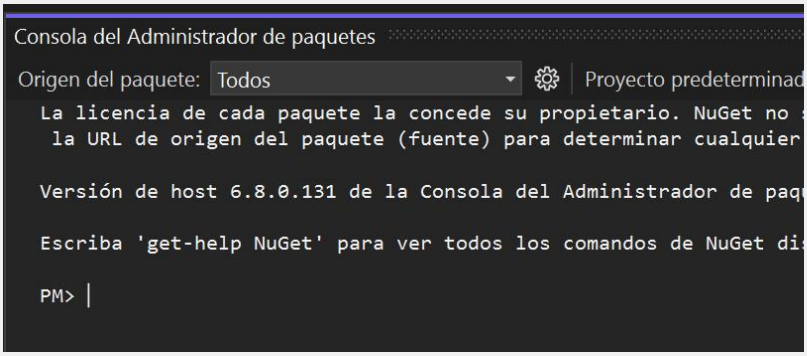
with NuGet Package 6.8.0

The first thing to do to have access to these methods is to install the ClosedXML package (version 0.102.2), which includes the DocumentFormat.OpenXml SDK, in the Nuget work console by clicking on Tools from the Visual Studio menu as follows:



and the work console will quickly appear at the bottom of the user window.
Visual Studio as shown below:

WordExcel;



And here it is written:

```
PM> Install-Package ClosedXML -Version 0.102.2
```

The first time, several lines will appear telling you how the installation process went; if the package was already installed in the project, the corresponding message is shown.

It is convenient to clarify that the ClosedXML package is installed within the solution that will use it, so every time you want to use said library in another solution, we must repeat these previous steps within said solution, unless it is available of a .DLL library that contains these Packages already incorporated and a set of methods created for this purpose, as is our case, since in the WordExcel library, there are several methods that allow you to easily work with Word and Excel, among many others. things.

The associated using is then declared, as follows:

```
//================================================================
using WE; // Word and Excel methods of the WE library
//================================================================
```

And now we are ready to use the methods that we will describe below
belonging to the WordExcel library.
Important note:
Personally I have always thought that there is nothing more illustrative to learn than
show several examples. For this reason, in both cases, we accompany this documentation with some examples of use that will surely help you save time.
programming, this is a very important issue and I hope you agree with me.

Word methods

Method of Opening and Creating a Word Document

WE.WordDocOpenWordDoc(stringPathDoc)

Description:

OpenWordDocIt is the first method to execute when creating a Word document and performs its opening.

The PathDoc parameter of type string defines the Path of the document to be created and the name of the same.

At the end of the execution, the associated object is returned "**ObjDoc**", which will be referenced
from this moment on by the remaining associated methods.

"**ObjDoc**"is an object of class"**WE.WordDoc**".

This returned object is also saved in a buffer that is a global variable
"Document"of WordExcel for future uses.

Example:
This example creates a new document "D:\AAA.docx and the object
returned from type WE.WordDoc has been called ObjDoc which is copied
also

```
WE.WordDoc      ObjDoc = OBJ.OpenWordDoc(@"D:\AAA.docx");
WE.WordDoc      DocBuffer = ObjDoc;
```

Important note:
The previous sequence could also have been written in the following form:

```
varObjDoc = OBJ.OpenWordDoc(@"D:\AAA.docx");
```

using the generic var data type.

Input Parameters:

stringPathDoc(defines the Path of the document to create)

Output parameters:
Does not have.

Return:
WE.WordDoc ObjDoc

Method of Creating a Heading of type Title1, Title2, etc.

```
public voidAddTitleType(WE.WordDocDocName,
 stringTitleTXT,
 WE.HeadingTypeTitleType,
 stringTitleFontType,
 intTitleFontSize,
 boolTitleBoldType,
 boolTitleItalicType,
 System.Drawing.ColorTitleColor,
 WE.AlignmentTitle Alignment)
```

Description:

The "AddTitleType" method of type "void" is used to create a paragraph of type Title1, 2, etc.

It contains 9 parameters:
"DocName" of class "WE.WordDoc" defines the name of the Word document returned in OpenWordDoc.
"TitleTXT" of type string, defines the text to be displayed.
"TitleType" is an object of the WE.HeadingType class that defines whether the given text will be in Title1 (Heading1), Title2 (Heading2), etc.
"TitleFontType" of type string, defines the font type, for example "Arial".
"TitleFontSize" of type int, defines the size of the font, for example 14.
"TitleBoldType" of type bool, if true means that "bold" letter is desired.
"TitleItalicType" of type bool, if true means that "Italic" letter is desired.
"System.Drawing.Color TitleColor", defines the color of the font, for example:

System.Drawing.Color.**Gray**

defines red color, if we choose System.Drawing.Color.**DarkRed** then it is dark red.
"WE.Alignment TitleAligment", defines the alignment of the footer, for
For example, if we write "**WE.Alignment.center**", then the text will appear centered.

Input parameters:

WE.WordDocDocName,
stringTitleTXT,
WE.HeadingTypeTitleType,
stringTitleFontType,
intTitleFontSize,
boolTitleBoldType,
boolTitleItalicType,
System.Drawing.ColorTitleColor,
WE.AlignmentTitle Alignment)

Output Parameters:
Does not have

Return:
Does not have

Example:

```
ObjDoc.AddTitleType(ObjDoc,
                    "PARAGRAPH 1",
                    WE.HeadingType.Heading1,
                    "Arial", 16, false, true, System.Drawing.Color.Black,
WE.Alignment.both);
```

In this case, a paragraph is added from the current line with the text:
PARAGRAPH1, of type Heading1 (Title1), with Arial font size 16, without bold font (Bold) of Italic font type, with font color in black and aligned on both sides.

Word Document Closing Method

Description:

It is the last method to execute when creating a Word document and it is the one that creates and
saves said document to disk.
After the "CloseWordDoc" has been executed, the use of the remaining methods is not allowed
on the current document, unless a new document is created again.
The DocOpen parameter of type "**WE.WordDoc**"defines the name that was
returned by the method "**OpenWordDoc**".

Note: Internally what is executed is the "Save" method of WE.WordDoc, which saves the document.

Format:
void**CloseWordDoc**(WE.WordDocDocOpen)

Input Parameter:
WE.WordDocDocOpen
where DocOpen is the name that was returned by**OpenWordDoc**

Output Parameter:
Does not have

Example:
OBJ.CloseWordDoc(ObjDoc);

Method of Inserting an Image into the Word Document

void InsertImageWordDoc(WE.WordDoc DocOpen, string Image)

Description:
The "InsertImageWordDoc" method is intended for inserting an image (.jpg, .bmp, etc.), in the current line, within the document creation process and requires two mandatory parameters:

1) "DocOpen" of type "WE.WordDoc" which is the name that was returned by "OpenWordDoc".

2) The second operand is given the path of the image of type "string".
It is convenient to clarify that at the end a string line "***" is written to avoid later conflicts in Ashampoo Office Write, which is an equivalent to Microsoft's Word but cheaper. This occurs when a table is created immediately after another table, which causes the following paragraph not to be displayed in said application. Windows 10 WordPad also detects the problem but warns that this could happen in apps equivalent to Microsoft Word. This bug will presumably be fixed in some Service Pack of the app.

Input parameters:
DocOpen
Image

Output parameters:
Does not have

Example of use:
```
OBJ.InsertImageWordDoc(ObjDoc,@"D:\LandscapePhoto.jpg");
```

Table Creation Method

```
WE.WordTableAppendTable(WE.WordDoc      DocName,
                        intTableRows,
                        intTableColumns)
```

Description:
The method**"AppendTable"**performs the creation of an empty table in Word on the line in course, so that it only takes into account the number of rows and columns in it.
This method has three input parameters:
1) DocName which is the name that was returned by "**"OpenWordDoc"**of the kind **WE.WordDoc**
2) TableRows is a numeric parameter of type "int" that defines the number of rows in the table. board.
3) TableColumns is a numeric parameter of type "int" that defines the number of columns of the table.
At the end of the execution, the internal name of the table is returned, which is returned with the type of the class**"WE.WordTable"**and which is the one that must be used later in all the methods associated with working with Docx tables.
Input parameters:
DocName
TableRows
TableColumns

Output parameters:
Does not have

Return: The name of the table in the format:
WE.WordTableBOARD

Columnar Header Creation Method

```
void PutHeaderName(WE.WordTable TableName,
                   int TableCell,
                     string ColumnHeader,
                     string ColumnHeaderFontType,
                     int ColumnHeaderFontSize,
                     System.Drawing.Color ColumnHeaderColor)
```

Description:

The method**"PutHeaderName"**of type "void" is the one that is responsible for writing each title of header in each cell (Column) of row 0 (Header Row).

It contains 6 parameters:

"TableName"of type (WE.WordTable)
"TableCell"of type (int)
"ColumnHeader"of type (string)
"ColumnHeaderFontType"of type (string)
"ColumnHeaderFontSize"of type (int)
"ColumnHeaderColor"of type (System.Drawing.Color)

Row cells are written center-aligned.
In**"TableName"**the name returned by the "AppendTable" method appears as the table name
In**"TableCell"**the column number of the Columnar Header (int) is given.
In**"ColumnHeader"**the column title (string) is defined.
In**"ColumnHeaderFontType"**defines the letter to use for the ColumnHeader (string).
In**"ColumnHeaderFontSize"**The font size (int) is defined.
In**"ColumnHeaderColor"**defines the color of the letter (System.Drawing.Color).
The number of the columns (Cells) are relative to 0 as are the numbers of the rows (Rows).

Input parameters:

"TableName" of type (WE.WordTable)
"TableCell" of type (int)
"ColumnHeader" of type (string)
"ColumnHeaderFontType" of type (string)
"ColumnHeaderFontSize" of type (int)
"ColumnHeaderColor" of type (System.Drawing.Color)

Output parameters:
Does not have.

Return:
Does not have

Header Creation Method with Italic Letters

This method is similar to the previous one with the small difference that the column titles are created with letter ***Italics***.

Cell Content Writing Method

```
void PutInfoCell(WE.WordTable TableName,
                 string TableRowInfo,
                  int TableRow,
                  int TableCell)
```

Description:
The "PutInfoCell" method of type "void" is the one that deals with writing the content of each cell one by one in a given row.

It contains 4 parameters:

"TableName"	of type (WE.WordTable)
"TableRowInfo"	of type (string)
"TableRow"	of type (int)
"TableCell"	of type (int)

Row cells are written left aligned.
In **"TableName"** The name that was given to the table using the "WE.WordTable" class appears
In **"TableRowInfo"** The content that you want to write in the cell in string format is specified.
In **"TableRow"** The number of the row containing the cell to be written is given.
In **"TableCell"** The number of the cell (column) is given where the desired content will be written.

The number of the columns (Cells) are relative to 0 as are the numbers of the rows (Rows).

Input parameters:

"TableName" of type (WE.WordTable)
"TableRowInfo" of type (string)
"TableRow" of type (int)
"TableCell" of type (int)

Output parameters:

Does not have.

Return:

Does not have.

Header and Footer Methods

```
void            CreateDocHeader(WE.WordDocDocName,
 string         HeaderTxt,
 string         HeaderFontType,
 int            HeaderFontSize,
 bool           HeaderBoldType,
 bool           HeaderItalicType,
 System.Drawing.Color           HeaderColor,
 WE.Alignment   HeaderAligment)
```

Description:

The "CreateDocHeader" method of type "void" is only used to create page headers if desired.

It contains 8 parameters:
"DocName" of class "WE.WordDoc" defines the name of the Word document.
"HeaderTxt" of type string, defines the header text of the pages.
"HeaderFontType" of type string, defines the font to use, for example "Arial".
"HeaderFontSize" of type int, defines the size of the font, for example 14.
"HeaderBoldType" of type bool, if true means that "bold" letter is desired.
"HeaderItalicType" of type bool, if true it means that the letter "Italic" is desired.
"System.Drawing.Color HeaderColor", defines the color of the letter, for example:
System.Drawing.Color.Griddefines red color, if we choose:
System.Drawing.Color.DarkRedthen it is dark red.
"WE.Alignment HeaderAligment", defines the alignment of the header text, for example, if we write "WE.Alignment.center", then the text will appear centered.
Note: Usually the colors chosen for this purpose are attenuated (not dark).

Input parameters:
WE.WordDocDocName,
stringHeaderTxt,
stringHeaderFontType,
intHeaderFontSize,
boolHeaderBoldType,
boolHeaderItalicType
System.Drawing.ColorHeaderColor,
WE.AlignmentHeaderAligment)

Output parameters:
Does not have.

Return:
Does not have.

Footer Creation Method

```
void    CreateDocFooter(WE.WordDocDocName,
 string FooterTXT,
 string FooterFontType,
 int    FooterFontSize,
 bool   FooterBoldType,
 bool   FooterItalicType,
 System.Drawing.Color          FooterColor,
 WE.Alignment       Footer Alignment)
```

Description:
The method "CreateDocFooters"of type"void" is only used to create the footer if desired.
It contains 8 parameters:
"**DocName**" of class "WE.WordDoc" defines the name of the Word document.
"FooterTXT" of type string, defines the footer text; the number of each page is written automatically right after this text.
"**FooterFontType**" of type string, defines the font type, for example "Arial".
"**HeaderFontSize**" of type int, defines the size of the font, for example 14.
"**HeaderBoldType**" of type bool, if true means that "bold" letter is desired.
"**HeaderItalicType**" of type bool, if true it means that the letter "Italic" is desired.
"**System.Drawing.Color FooterColor**", defines the color of the letter, for example:
System.Drawing.Color.Red defines red color, and if we choose:
System.Drawing.Color.DarkRed then it is dark red.
"**WE.Alignment FooterAligment**", defines the alignment of the foot of
page, for example, if we write "WE.Alignment.center", then the text will appear centered.
Note: Usually the colors chosen for this purpose are attenuated (not dark).

Input parameters:
WE.WordDocDocName,
stringFooterTXT,
stringFooterFontType,
intFooterFontSize,
boolFooterBoldType,
boolFooterItalicType,
System.Drawing.ColorFooterColor,
WE.AlignmentFooter Alignment)

Output parameters:
Does not have.

Return:
Does not have.

Method of Adding a Paragraph to the Document

voidAddParagraph(WE.WordDocDocName,
stringParagraphTXT,
stringParagraphFontType,
intParagraphFontSize,
boolParagraphBoldType,
boolParagraphItalicType,
System.Drawing.ColorParagraphColor,
WE.AlignmentParagraph Alignment)

Description:
The "AddParagraph" method of type "void" is used to add a new paragraph to the document.
It contains 8 parameters:
"DocName" of the class "WE.WordDoc" defines the name returned in the
document creation.
"ParagraphTXT" of type string, defines the text of the paragraph to write.
"ParagraphFontType" of type string, defines the font to use in the paragraph,
for example "Arial".
"ParagraphFontSize" of type int, defines the size of the font, for example 12.
"ParagraphBoldType" of type bool, if true means that "bold" letter is desired.
"ParagraphItalicType" of type bool, if true means that letter is desired
"Italic".
"System.Drawing.Color ParagraphColor", defines the color of the letter, for example
System.Drawing.Color.Griddefines red color, and if we choose:
System.Drawing.Color.DarkRedthen it is dark red.
"WE.Alignment ParagraphAligment", defines the alignment of the paragraph, for example, if we write
"WE.Alignment.center", then the text will appear centered.

Input parameters:
WE.WordDocDocName,
stringParagraphTXT,
stringParagraphFontType,
intParagraphFontSize,
boolParagraphBoldType,
boolParagraphItalicType,
System.Drawing.ColorParagraphColor,
WE.AlignmentParagraph Alignment)

Output parameters:
Does not have.

Return:
Does not have.

Example Program for Creating a Word Document

The best way to understand a set of new methods related to a specific topic is to see an example of using the code and then see the result.

Below is a simple program that uses all of the above methods.
This simple program is of the WinForms type with a single button that, when clicked, executes the program.

```
usingWE;
usingSystem.Windows.Forms;


namespaceTestExcelNuget
{
 public partial class Form1:Form
{
WE.WordDoc        Document;
 public Form1()
{
InitializeComponent();
}

 private voidbutton1_Click(objectssender, System.EventArgs e)
{
WordExcel OBJ =newWordExcel(); // The OBJ object is instantiated to use all the methods of the WordExcel library

 varObjDoc = OBJ.OpenWordDoc(@"D:\AAA.docx");
Document = ObjDoc;
 //==========================<HEADERS CREATION>================ =======
OBJ.CreateDocHeader(ObjDoc,"Document Header","Aryan", 14,true,true, System.Drawing.Color.Gray,
WE.Alignment.right);
 //======================<CREATION OF THE FOOTERS>=================== =======
OBJ.CreateDocFooter(ObjDoc,"Page # ","Aryan", 14,true,true, System.Drawing.Color.DarkGray,
WE.Alignment.right);
```

```
 //======================< CREATION OF PARAGRAPH1>====================== =======
OBJ.AddTitleType(ObjDoc,"PARAGRAPH 1", WE.HeadingType.Heading1,"Aryan", 16,false,true,
                                    System.Drawing.Color.Black,
WE.Alignment.both);
OBJ.AddParagraph(ObjDoc,"This is the demonstrative text of the first test paragraph of creating a Word document using the program."+
 "Here we aim to show the relative ease with which a Word document is created with this set of methods.",
 "Aryan",
10,false,true, System.Drawing.Color.DarkRed, WE.Alignment.both);
OBJ.AddParagraph(ObjDoc,"This is the second demonstrative paragraph of this example program, it is convenient to clarify that our
objective"+
 "The main thing is to simplify the programming of Word-oriented applications as much as possible.",
 "Aryan",
10,false,false, System.Drawing.Color.DarkGreen, WE.Alignment.both);
 //---------------------------------------- ---------------------------------------- ---------
 varTABLE = OBJ.AppendTable(ObjDoc, 3, 3);
 //---------------------------------------- ---------------------------------------- ---------
OBJ.PutHeaderName(TABLE, 0,"NAME","Aryan", 14, System.Drawing.Color.Black);
OBJ.PutHeaderName(TABLE, 1,"AGE","Times New Roman", 14, System.Drawing.Color.DarkRed);
OBJ.PutHeaderName(TABLE, 2,"SEX","Courier New", 14, System.Drawing.Color.DarkMagenta);
 //---------------------------------------- ---------------------------------------- ---------
OBJ.PutInfoCell(TABLE,"HECTOR",1,0);
OBJ.PutInfoCell(TABLE,"75",1,1);
OBJ.PutInfoCell(TABLE,"M",1, 2);
OBJ.PutInfoCell(TABLE,"IRMA",2,0);
OBJ.PutInfoCell(TABLE,"60",2,1);
OBJ.PutInfoCell(TABLE,"F",2, 2);
 //---------------------------------------- ---------------------------------------- ---------
ObjDoc.InsertTable(TABLE);
 //---------------------------------------- ---------------------------------------- ---------
OBJ.AddParagraph(ObjDoc,"AAA.","Aryan", 10,false,true, System.Drawing.Color.DarkRed,
WE.Alignment.both);
OBJ.AddParagraph(ObjDoc,"BBB.","Aryan", 10,false,false, System.Drawing.Color.DarkRed,
WE.Alignment.both);
 //---------------------------------------- ---------------------------------------- --------
 varTABLE1 = OBJ.AppendTable(ObjDoc, 4, 3);
 //---------------------------------------- ---------------------------------------- --------
OBJ.PutInfoCell(TABLE1,"Elvis",1,0);
OBJ.PutInfoCell(TABLE1,"17",1,1);
OBJ.PutInfoCell(TABLE1,"M",1, 2);
OBJ.PutInfoCell(TABLE1,"Gretel",2,0);
OBJ.PutInfoCell(TABLE1,"40",2,1);
OBJ.PutInfoCell(TABLE1,"F",2, 2);
OBJ.PutInfoCell(TABLE1,"Brandon",3,0);
OBJ.PutInfoCell(TABLE1,"1",3,1);
OBJ.PutInfoCell(TABLE1,"M",3, 2);
 //---------------------------------------- ---------------------------------------- ---------
OBJ.PutHeaderNameItalic(TABLE1, 0,"NAME","Aryan", 18, System.Drawing.Color.Red);
OBJ.PutHeaderNameItalic(TABLE1, 1,"AGE","Aryan", 18, System.Drawing.Color.Red);
OBJ.PutHeaderNameItalic(TABLE1, 2,"SEX","Aryan", 18, System.Drawing.Color.Red);
ObjDoc.InsertTable(TABLE1);
 //---------------------------------------- ---------------------------------------- ---------
OBJ.AddParagraph(ObjDoc,"XXX.","Aryan", 10,false,true, System.Drawing.Color.DarkRed,
WE.Alignment.both);
OBJ.AddParagraph(ObjDoc,@"https://www.sunpass.com/","Aryan", 10,false,true,
                                    System.Drawing.Color.DarkRed, WE.Alignment.left);
 //---------------------------------------- ---------------------------------------- ---------
OBJ.InsertImageWordDoc(ObjDoc,@"D:\LANDSCAPE.jpg");
OBJ.AddParagraph(ObjDoc,"XXX.","Aryan", 10,false,true, System.Drawing.Color.DarkRed,
WE.Alignment.both);

ObjDoc.ReplaceText("AAA.","AAA replacement sequence.");
```

```
ObjDoc.ReplaceText("BBB.","BBB replacement sequence.");
ObjDoc.ReplaceText("XXX.","Mr. XXX, from now on you will be called YYY.");

OBJ.CloseWordDoc(ObjDoc);
Application.Exit(); // End of the process
}
}
}
```

Generated .docx document

Below is the content of the generated Word document ("AAA.docx")

PARAGRAPH 1

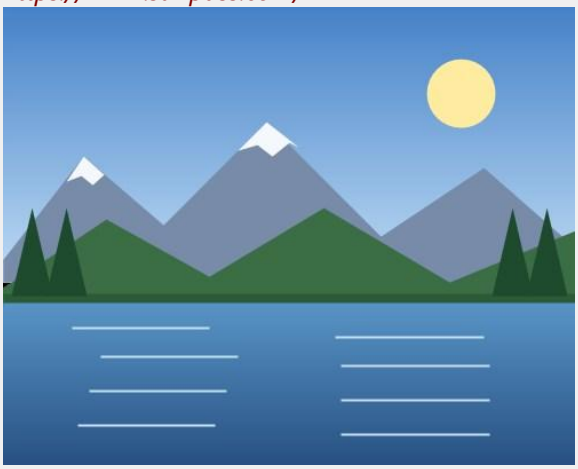
This is the demonstrative text of the first test paragraph of creating a Word document through the program. Here we aim to show the relative ease with which a Word document is created with this set of methods.
This is the second demonstrative paragraph of this example program. It is worth clarifying that our main objective is to simplify the programming of Word-oriented applications as much as possible.

NAME	AGE	SEX
HECTOR	75	M
IRMA	60	F

AAA replacement sequence.
BBB replacement sequence.

NAME	*AGE*	*SEX*
Elvis	17	M
Gretel	40	F
Brandon	1	M

Mr. XXX, from now on you will be called YYY.
https://www.sunpass.com/



***

Mr. XXX, from now on you will be called YYY.
This simple example shows how the Word methods work, from the WordExcel library.

Excel methods

ExcelCreateDocExcel() Method for Creating an EXCEL Document with its Initial Sheet

==========

Format:

XLWorkbook ExcelCreateDocExcel(string DocExcel, string SheetName)

Description:

The ExcelCreateDocExcel() method is only used to create a new EXCEL document
and immediately after its execution the method must be executed:
XLWorkbook ExcelOpenFile(string PathFile)
This is mandatory as long as you want to execute actions specific to the definition
of all the characteristics that we want to add to said EXCEL document, such as
the heading titles of each column, the colors of a cell (Cell), the type
font size of a cell, size, and so on.
At the end of the execution of this method, two important names are deposited for the
I work with EXCEL in two global variables accessible by the programmer:
"ExcelFile" -> Contains the internal name to be used by the different methods
and that is associated with the EXCEL document that is being generated.
"ExcelSheet" -> It is the name of the sheet that contains the table of rows and
columns with which to work.
Important note:
If, after executing the ExcelCreateDocExcel method, you want to use the
different existing methods for creating a given spreadsheet,
then the ExcelOpenFile method must be executed, otherwise
a program exception occurs.
This is because at the end of the ExcelCreateDocExcel code a statement is executed
workbook.SaveAs(DocExcel);
that records the new document on disk, closing it and leaving the sheet
Excel is empty, so that it can later be defined using the different
methods that will give the desired format. For this reason it should usually be executed
then the corresponding ExcelOpenFile that will open the doors to the programmer to
be able to use any of the methods intended for this purpose.

Input parameters:
1) string DocExcel -> string that defines the Path and the desired file name
Example: @"D\DocumentosExcel\Alumnos.xlsx"
2) string SheetName -> string that defines the name of the Excel sheet that we are going to
fill with the different methods defined for this purpose.
Output parameters:
The internal name of the Excel document to be used in various method parameters
defined. This name is from the Data Type "XLWorkbook".

==========

METHOD ExcelOpenFile(string PathFile)

(Opening an Excel Document)

==========

Format:
XLWorkbook ExcelOpenFile(string PathFile)

Description:
The ExcelOpenFile(string PathFile) method is intended to open the blank document created above by the "ExcelCreateDocExcel" method. This is mandatory in order to avoid an exception that will cancel your task.

Input parameters:
1) string PathFile -> string that defines the Path and the name of the file created in the step former.
Example: @"D\DocumentosExcel\Alumnos.xlsx"
Output parameters: None
Return:
The internal name of the Excel document to be used in various method parameters defined. This name is of the Data Type "XLWorkbook" and is required by several the methods to use in generating the document.
If any error is detected in the opening, the corresponding message is issued with the exception detected.

==========

ExcelSaveFile() METHOD TO SAVE THE EXCEL DOCUMENT

==========

Format:
void ExcelSaveFile(string FileName)

Description:
The "void ExcelSaveFile(string FileName)" method is used to save the document generated by program with the Path and the name of the file given in the operand string FileName.
Important note:
This method finishes the work with the document that is generated, so if you want within the same program to continue working, it is mandatory to use the method again "ExcelOpenFile", otherwise an exception is thrown.

Input parameters:
1) string FileName -> string that defines the Path and the name of the file created in the step former.
Example: @"D\DocumentosExcel\Alumnos.xlsx"
Output parameters: None

Return:
Does not have (void)

==========

ExcelFormatTitles() METHOD OF WRITING A COLUMN TITLE (Row = 1, Column = n)

==========

Format:

void ExcelFormatTitles(int ColumnCell,
string ValueCell,
string FontName,
int FontSize,
bool FontBold,
bool FontItalic,
XLColor BackColor, XLColor ForeColor)

Description:

The titles of each column defined by the user in EXCEL can be varied and with
a style format defined by the programmer. Since such column headings must
appear on line 1 (row = 1) since it is not required as a parameter and for this reason the
The first thing that appears is an integer that defines the desired title column.

Input parameters:
ColumnCell - Column number.
ValueCell - It is a string that defines the column title.
FontName - It is a string that defines the font to be used ("Arial", "Courrier New", etc.
FontSize - The size of the font (12, 16, etc.)
FontBold - Bool type variable that if it is true means that you want bold font.
FontItalic - Bool type variable that if it is true means that you want an Italic font.
BackColor - XLColor class variable that defines the color of Background
ForeColor - XLColor class variable that defines the Foreground color (letter)

Output parameters: None

Return: Does not have (void)
If any error is detected in the opening, the corresponding message is issued with the
exception detected.

==========

ExcelFormatCell() METHOD OF FORMATTING A CELL

==========

Format:

void ExcelFormatCell(int RowCell,
int ColumnCell,
string ValueCell,
string FontName,
int FontSize,
bool FontBold,
bool FontItalic,
XLColor BackColor, XLColor ForeColor**)**

Description:

The "ExcelFormatCell" method is used to assign a value to a cell under the format of (row, column) with color style, font, font size, etc.
Input parameters:
RowCell - Row number.
ColumnCell - Column number.
ValueCell - It is a string that defines the value of the column.
FontName - It is a string that defines the font to be used ("Arial", "Courrier New", etc.
FontSize - The size of the font (12, 16, etc.)
FontBold - Bool type variable that if it is true means that you want bold font.
FontItalic - Bool type variable that if it is true means that you want an Italic font.
BackColor - XLColor class variable that defines the cell's Background color
ForeColor - XLColor class variable that defines the cell's Foreground color
Output parameters: None
Return: Does not have (void)
If any error is detected in the opening, the corresponding message is issued with the exception detected.

==========

ExcelWriteCellRC() METHOD OF WRITING A CELL IN AN EXCEL DOCUMENT

==========

Format:

void ExcelWriteCellRC(int RowCell,
int ColumnCell,
string ValueCell)

Description:

The "ExcelWriteCellRC" method is used to assign a value to a cell under the format of (row, column).

Input parameters:
RowCell - Row number.
ColumnCell - Column number.
ValueCell - It is a string that defines the value of the column.

Output parameters: None

Return: Does not have (void)
If any error is detected in the opening, the corresponding message is issued with the exception detected.

==========

ExcelWriteCell() METHOD OF WRITING A CELL OF THE EXCEL DOCUMENT

==========

Format:

void ExcelWriteCell(string InputCell,
string ValueCell)

Description:

The "ExcelWriteCell" method is used to assign a value to a cell under the format of (string).
Input parameters:
InputCell - Output cell in string format, example "B2", "C5", etc.
ValueCell - It is a string that defines the value that the column will have.
Output parameters: None
Return: Does not have (void)
If any error is detected in the opening, the corresponding message is issued with the exception detected.

==========

ExcelGetCellRC() METHOD OF WRITING A CELL WITH A NAME IN string

==========

Format:

void ExcelGetCellRC(int RowCell, int ColumnCell)

Description:

The "ExcelGetCellRC" method is used to obtain the value to a cell in the format (R, C)

Input parameters:
RowCell - Row number of the Input Cell.
ColumnCell - Column number of the input Cell.

Output parameters: None

Return: Does not have (void)

If any error is detected in the opening, the corresponding message is issued with the exception detected.

==========

""ExcelGetCell" METHOD FOR READING A CELL OF THE EXCEL DOCUMENT

==========

Format:

string ExcelGetCell(string InputCell)

Description:

The "ExcelGetCell" method is used to obtain the value to a cell in the string format of Excel.
Input parameters:
InputCell - string that references the Input Cell, e.g., "A4", "B2", etc.
Output parameters: None
Return: Returns the cell value in string format.
If any error is detected in the opening, the corresponding message is issued with the exception detected.

==========

==========

ExcelGetCellDataTypeRC() METHOD OF READING THE DATA TYPE OF A CELL (r, c)

==========

Format:
string ExcelGetCellDataTypeRC(int RowCell,
int ColumnCell)

Description:

The "ExcelGetCellDataTypeRC" method is used to obtain the data type of a cell under Excel (r, c) format.

Input parameters:
RowCell - int that gives the row value of the input Cell.
ColumnCell - int that gives the column value of the input Cell.

Output parameters: None

Return: Returns the data type of the Excel cell in string format.

If any error is detected in the opening, the corresponding message is issued with the exception detected.

==========

ExcelGetCellDataType() METHOD FOR READING THE DATA TYPE OF A CELL IN "string" FORMAT

==========

Format:
string ExcelGetCellDataType(string InputCell)

Description:
The "ExcelGetCellDataType" method is used to obtain the data type of a cell under Excel string format.
Input parameters:
InputCell - Address of the input cell in EXCEL string format ("A4", "B6", etc.)
Output parameters: None
Return: Returns the data type of the Excel cell in string format.
If any error is detected in the opening, the corresponding message is issued with the exception detected.

==========

ExcelFormula() METHOD TO DESCRIBE A FORMULA IN EXCEL

==========

Format:
void ExcelFormula(int CellDef, string Formula)

Description:
Method to define an Excel formula.

Input parameters:
ValueCell - It is a string that defines the column title.
Formula - It is a string that defines the font to be used ("Arial", "Courrier New", etc.

Output parameters: None

Return: Does not have (void)

If any error is detected in the opening, the corresponding message is issued with the exception detected.

==========

Example program for generating an EXCEL document

```
using ClosedXML.Excel;
using DocumentFormat.OpenXml.Vml;
usingSystem;
using System.Windows.Forms;
namespace TestExcelNuget
{
//================================================================
// Author: Lic. Hector Fortun (January 22, 2024)
//================================================================
// Example program for creating and modifying documents
// EXCEL under the use of the methods added to the library
// WordExcel (Library New) created by me to work with concepts
// which usually consume a lot of programming time.
// This library covers several job profiles of different
// type such as, for example, information validation methods,
// creation and modification of MS Word documents or similar
// Ashampoo Writer, methods for messaging, working with date and time,
// calls to Windows procedures and objects, Error Messages
// or Informative, etc. All of the above, always with the objective
// main goal of making difficult things easy and saving time.
//
// The main objective has been to create a simple logic and to
// enough time to make it easier for the low C# programmer
// Visual Studio working with Excel documents, not only
// MSOffice, but also other similar products such as Office
// from Ashampoo that accepts .xlsx and xls spreadsheets.
//
// Important note:
//
// Since originally being able to program for MSExcel required
// that MSOffice was installed on your PC by using a library
// required by Microsoft (Interop), since we decided to rely on
// some Nuget Packages, such as DocumentFormat.OpenXml (for Word) and
// ClosedXML (for EXCEL), both under the MIT license.
// Given my personal experience with the previous ones, I observed that
// they were a little reluctant to break the inertia and I decided to structure these
// tools in a very simple way with new methods that
// will help the programmer to advance quickly in the setup
// quick of the most everyday tasks.
// However, we have left the doors open for
// when the programmer decides, use the tools
// original from the manufacturer, to whom we are very grateful for
// have the idea of        being able to work with equivalents to Word and Excel
// from Microsoft without having to have them installed on our PC.
// It is convenient to clarify that both packages allow free use, also
// in commercial applications.

//================================================================
```

```
 public partial class Form1 : Form
{
//---------------------------------------------------------
// Global Variables For Excel
//---------------------------------------------------------
XLWorkbook ExcelFile = null;
IXLWorksheet ExcelSheet = null;
//---------------------------------------------------------
// The object named is then instantiated
// OBJ with the WordExcel class of the multipurpose library
// WordExcel. For this reason, reference will be made to this object
// frequently within the program.
//---------------------------------------------------------
WordExcel OBJ = new WordExcel();
//---------------------------------------------------------
int C = 0;
int F = 0;
string B2 = "";
string B3 = "";
string B4 = "";
string cell = "";
float B2Float = 0;
float B3Float = 0;
float B4Float = 0;
float B6Float = 0;
string DataType = "";
XLDataType XX = new XLDataType();
//---------------------------------------------------------
public Form1()
       {
       InitializeComponent();
       }

private void button1_Click(object sender, System.EventArgs e)
{
try
{
// Opening of the new Excel document to be generated
// which in turn will contain the first sheet "Sheet1".

OBJ.ExcelCreateDocExcel(@"D:\YYYNew.xlsx", "Sheet1");

// Open the file created in the previous step to
// work with he.

OBJ.ExcelFile = OBJ.ExcelOpenFile(@"D:\YYYNew.xlsx");

// Open the spreadsheet (sheet) and assign it to
// ExcelSheet
OBJ.ExcelSheet = OBJ.ExcelOpenSheet(1);

OBJ.ExcelFormatTitles(1, "NAME", "Segoe UI", 18, true, true,
                    XLColor.Red, XLColor.Yellow);
OBJ.ExcelFormatTitles(2, "SALARY", "Segoe UI", 18, true, true,
                    XLColor.Red, XLColor.Yellow);
OBJ.ExcelFormatCell(2, 1, "Arial", "Arial", 18, true, false,
                    XLColor.DarkBlue, XLColor.Turquoise);
OBJ.ExcelFormatCell(3, 1, "Times New Roman", "Times New Roman",
                    16, true, false,
XLColor.Black, XLColor.White);
OBJ.ExcelFormatCell(4, 1, "Courrier New", "Courrier New",
                    14, true, false,
XLColor.Red, XLColor.Yellow);

OBJ.ExcelFormatCell(2, 2, "250.50", "Times New Roman",
                    18, true, false,
```

```
XLColor.Turquoise, XLColor.DarkBlue);
OBJ.ExcelFormatCell(3, 2, "300.48", "Arial", 16, true, false,
                    XLColor.White, XLColor.Black);
OBJ.ExcelFormatCell(4, 2, "600.44", "Segoe UI", 14, true, false,
                    XLColor.Yellow, XLColor.Red);

OBJ.ExcelSheet.Row(1).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(2).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(3).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(4).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(5).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(6).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

//=======ARITMETIC OPERATIONS WITH CELLS IN A COLUMN=======
B2 = OBJ.ExcelSheet.Cell("B2").Value.ToString();
B3 = OBJ.ExcelSheet.Cell("B3").Value.ToString();
B4 = OBJ.ExcelSheet.Cell("B4").Value.ToString();
//---------------------------------------------------------
B2Float = float.Parse(B2);
B3Float = float.Parse(B3);
B4Float = float.Parse(B4);
//---------------------------------------------------------------------
B6Float = B2Float + B3Float + B4Float;
//---------------------------------------------------------------------
OBJ.ExcelSheet.Cell("B6").Value = B6Float.ToString();

//--------------------FORMAT THE RESULT COLUMN----------------------
OBJ.ExcelFormatCell(6, 2, OBJ.ExcelSheet.Cell("B6").Value.ToString(),
                    "Arial", 18, true, false,
                    XLColor.DarkRed, XLColor.Yellow);
//------------------------------------------------ --------------------
// The result of this arithmetic operation "(B4 + B6 + A7) / 3"
// is in A8
//------------------------------------------------ --------------------
// It is convenient to note that cell B5 has no content,
// however it is accepted in the operation as if the value were zero.
//------------------------------------------------ --------------------
OBJ.ExcelFormula("A8", "(B4 + B5 + B6 + A7) / 3");

//==================ASSIGNING A DATA TYPE TO A CELL====================
OBJ.ExcelSheet.Cell(7,1).Value = "123.45";
B2 = OBJ.ExcelSheet.Cell(7,1).DataType.ToString();
XX = OBJ.ExcelSheet.Cell(7,1).DataType;




OBJ.ExcelFormatCell(7, 1, OBJ.ExcelSheet.Cell(7,1).Value.ToString(),
                    "Arial", 14, true, false,
```

```
                    XLColor.DarkRed, XLColor.Yellow);
cell = OBJ.ExcelGetCell("A7");
XX = XLDataType.Number;

if    (XX == XLDataType.Text)
       {
       OBJ.MensINFO("Cell A7 = Text");
       }
if    (XX == XLDataType.Number)
       {
       OBJ.MensINFO("Cell A7 = Number");
       }
//---------------------------------------------------------------------
OBJ.ExcelWriteCellRC(5, 1, "AAA");
OBJ.ExcelWriteCellRC(6, 1, "BBB");

OBJ.ExcelWriteCell("B3", "CCC");
OBJ.ExcelWriteCell("B2", "DDD");
// Get the cell value (3,2)
cell = OBJ.ExcelGetCellRC(3, 2);
//Get the Data type of (4,2)
DataType = OBJ.ExcelGetCellDataTypeRC(4, 2);
//Get the Data type of "B2"
cell = OBJ.ExcelGetCellDataType("B2");

OBJ.MensINFO("# Columns: " + C.ToString() + "\n" +
             "# Rows: " + F.ToString() +
             "\nCell(3,2) = " + cell +
             "\nDataType (4,2) = " + DataType +
             "\nData Type (2,2) = " + cell);

OBJ.MensINFO("End of Process");
}
catch
       {
       OBJ.MensINFO("Exception Detected - The program is cancelled\n" +
       e.ToString());
       Application.Exit();
       }
OBJ.ExcelSaveFile(@"D:\AAAOutput.xlsx");
Application.Exit();
}
}
}
```

==========

DataGridView METHODS

==========

DGVColumnSumarize() method

==========

FORMAT:

```
int DGVColumnSumarize(DataGridView DGV,
int DGVColumn,
ref double SUMMARY)
```

DESCRIPTION:

The DGVColumnAverageSelectedRows() method performs the calculation of the sum of the quantities
that are found in the cells of a column given by DGVColumn of the DataGridView of the first operand, leaving the result in the double type variable of the third operand SUMMARY, by reference.

Input parameters:

DataGridView DGV
int DGVColumn
ref double SUMMARY

Output parameters:

ref double SUMMARY -> Transmission by reference of the columnar sum

Return:

An integer value that reports how the execution of the method ended.

Example:

```
DGVColumnSumarize(DataGridView1, 4, refTotalSum);
```

==========

DGVColumnSumarizeSelectedRows() method

==========

FORMAT:

```
int DGVColumnSumarizeSelectedRows(DataGridView DGV,
int DGVColumn,
ref double SUMMARY)
```

DESCRIPTION:

The DGVColumnAverageSelectedRows() method performs the calculation of the sum of the quantities that are found in the selected cells of a column given by DGVColumn of the DataGridView of the first operand, leaving the result in the double type variable of the third operand SUMMARY, by reference.

Input parameters:

DataGridView DGV
int DGVColumn
ref double SUMMARY

Output parameters:

ref double SUMMARY -> Transmission by reference of the columnar sum

Return:

An integer value that reports how the execution of the method ended.

Example:

```
DGVColumnSumarizeSelectedRows(DataGridView1, 4, ref TotalSum);
```

==========

DGVColumnAverage() method

==========

FORMAT:

```
public int DGVColumnAverage(DataGridView DGV,
                            int DGVColumn,
                            ref double AVERAGE)
```

DESCRIPTION:

The DGVColumnAverage() method calculates the average of the quantities that are found in the cells of a column given by DGVColumn of the DataGridView of the first operand, leaving the result in the double type variable of the third operand AVERAGE, for reference.

Input parameters:

DataGridView DGV
int DGVColumn
ref double AVERAGE

Output parameters:

ref double AVERAGE -> Transmission by reference

Return:

An integer value that reports how the execution of the method ended.

Example:

```
DGVColumnAverage(DataGridView1, 4, refAverage);
```

==========

DGVColumnAverageSelectedRows() method

==========

FORMAT:

```
int DGVColumnAverageSelectedRows(DataGridView DGV,
                                 int DGVColumn,
                                 ref double AVERAGE)
```

DESCRIPTION:

The DGVColumnAverageSelectedRows() method calculates the average of the quantities found in the selected cells of a column given by DGVColumn of the DataGridView of the first operand, leaving the result in the double type variable of the third operand AVERAGE, by reference.

Input parameters:

DataGridView DGV
int DGVColumn
ref double AVERAGE

Output parameters:

ref double AVERAGE -> Transmission by reference

Return:

An integer value that reports how the execution of the method ended.

Example:

```
DGVColumnAverageSelectedRows(DataGridView1, 4, ref Average);
```

==========

==========

DGVGetCell() method

==========

FORMAT:

string DGVGetCell(DataGridView DGV,
int DGVRow,
int DGVColumn)

DESCRIPTION:

The DGVGetCell() method returns the content of the cell (DGVRow, DGVColumn) in format string

Input parameters:

DataGridView DGV
int DGVRow
int DGVColumn

Output parameters:

Does not have

Return:

Returns the content of the cell (DGVRow, DGVColumn) in string format.

Example:

DGVGetCell(DataGridView1, 4, 1);

Returns the content of cell (4,1) in string format.

==========

DGVPutCell() method

==========

FORMAT:

void DGVPutCell(DataGridView DGV,
string CellValue,
int DGVRow,
int DGVColumn)

DESCRIPTION:

The DGVPutCell() method writes the contents of CellValue to the cell (DGVRow, DGVColumn) of the DataGridView in string format.

Input parameters:

DataGridView DGV
string CellValue
int DGVRow
int DGVColumn

Output parameters:
Does not have
Return:
Does not have.
Example:
DGVPutCell(DataGridView1, "Irma", 4, 1);
Write the string "Irma" to cell (4,1) in string format.
==========